SIMATS ENGINEERING (SAVEETHA SCHOOL OF ENGINEERING)

Department of Computer Science and Engineering

ASSESSMENT TOOL 1 – ANALYTICAL PROBLEM SOLVING

Course Code:	CSA0309	**Course Title:**	Data Structures
CO Assessed:	CO2 – Manipulation (BL3, BL4)	**Assessment:**	Assessment Tool 1 – Analytical Problem Solving
Weightage:	30% of CIA	**Total Marks:**	100
CO2 Statement:	*Implement linear data structures such as stacks, queues, and linked lists, and analyze the efficiency of linear and binary search techniques.*		
Student Name:	R.DINESH	**Reg. No.:**	192524130
Section / Batch:	2YEAR	**Date:**	17/07/2026

Code of Conduct

I, R.DINESH (192524130) *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the Student: R. Dinesh

Instructions

- This test contains 80 Analytical problem-solving questions. Total: 100 Marks.
- When a student answer using an Analytical problem-solving, in evaluation the answer should be with following five requirements: Understanding of problem, select appropriate data structure, Design the solution, analyze, Clarity of representation.
- Each student should write 2 questions. No negative marking. Unanswered questions carry zero marks.
- No DTP printouts or electronic devices permitted. They should provide written materials.

Section / Topic	Focus	Q Nos	Marks
Linked Data Structure, Search Data Structure	List Operations, Binary Search	1	50
Stack Data Structure, Queue Data Structures	Stack Notations, Priority Queue	2	50
	GRAND TOTAL:	**100 Marks**	

Rubrics for Calculation

Criteria	Weightage	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Problem Understanding	20	Demonstrates complete and precise understanding; identifies all constraints and edge cases	Shows clear understanding with minor gaps	Partial understanding; misses some constraints	Misinterprets problem or lacks clarity
Data Structure Selection	20	Selects optimal data structure with strong justification and comparison	Selects appropriate structure with reasonable justification	Selects somewhat suitable structure with weak reasoning	Incorrect or no data structure selection
Algorithm Design	20	Designs efficient, logically sound, and well-structured algorithm	Algorithm is correct with minor inefficiencies	Algorithm is partially correct or lacks clarity	Algorithm is incorrect or missing
Accuracy of Solution	30	Error-free, well-structured, optimized code/solution	Minor errors but overall functional	Multiple errors; partially working	Non-functional or missing solution
Clarity	10	Exceptionally clear, well-organized, uses diagrams effectively	Clear and organized with minor issues	Somewhat organized but lacks clarity	Poorly organized and unclear

Faculty In-charge	Course Coordinator	Program Director
Signature:	Signature:	Signature:
Date:	Date:	Date:

1) Insert the elements 100, 200, 300, 400, 500 into the linked stack and perform two pop operations. Show the linked structure before and after the operations.

Problem Understanding:-

The objective is to implement a stack using a linked list. A stack follows the Last in first out (LIFO) principle, where the last inserted element is the first to be removed. The problem required inserting five elements into the stack and then performing two pop operations while showing the stack before and after deletion.

Data Structure Selection:-

A linked list is choosen because it supports dynamic memory allocation and efficient insertion and deletion at the top of the stack.

Justification:-

Criteria	Linked stack	Reason.
Memory	Dynamic	No fixed size limitation
Push	O(1)	Insert at Top
Pop	O(1)	Delete from Top
Flexibility	High	Memory allocated as needed.

Analysis:- Compared to an-array based stack, a linked stack avoids overflow due to fixed capacity and is more suitable when the number of elements in unknown.

Algorithm:-

Push:-

1) Create a new node.
2) Store the data
3) Point the new node to current TOP
4) Update TOP to the new node.

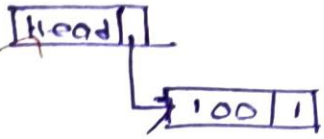
POP:-

1) Check whether the stack is empty.
2) Store the TOP node.
3) Move TOP to the next node.
4) Delete the previous TOP node.

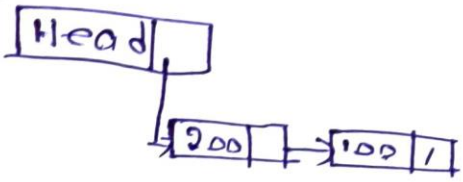
Execution:-

Initial stack : TOP = NULL

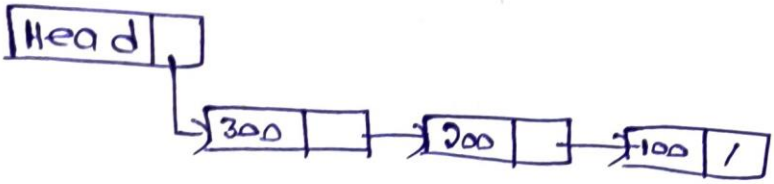
Push 100:



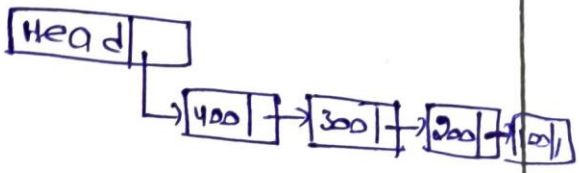
Push 200:



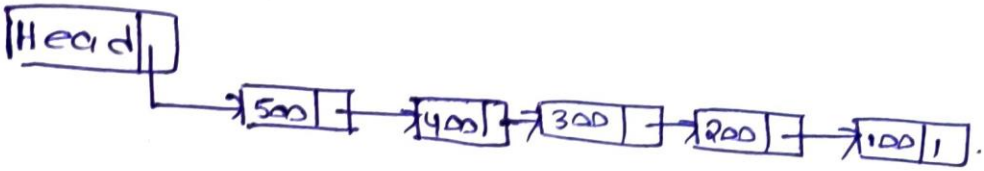
Push 300:



Push 400:



Push 500:



Pop Operations:

First pop :- 500.

TOP
↓
400 → 300 → 200 - 100 → Null.

Second pop :- 400.

TOP
↓
300 → 200 → 100 → NULL

Advantages:-

- Dynamic memory allocation.
- Efficient insertion and deletion.
- No shifting of elements.
- suitable for applications with unpredictable data size.

Applications:-

- Function call management
- Undo/redo operations
- Browse history
- Expression evaluation.
- Depth first search (DFS)

Conclusion:-

A linked stack efficiently implements the LIFO principle using dynamic memory allocation. After inserting the given elements, the first two pop operations remove 500 and 400 leaving 300, 200, 100 in the stack. The implementation provides O(1) time complexity for both push and pop operations, making it suitable for applications that require fast and dynamic stack operations.

2) A circular Queue has a size of 6 following operations: Enqueue(10), Enqueue(20), Enqueue(30), Dequeue(), Enqueue(40), Enqueue(50), Enqueue(60), Enqueue(70).

Problem Understanding:-

A circular queue is a linear data structure that follows the FIFO (First in first out) principle. Unlike a linear queue, the last position is connected to the first position, allowing the queue to reuse vacant spaces created after deletions.

In this problem, a circular queue of size 6 is used to perform enqueue and dequeue operations while tracking the Front and rear pointers after every operation.

Analysis:-

After a dequeue operation, one position becomes empty. In a circular queue, this empty position is reused when the rear reaches the end of the array. This makes the circular queue more memory efficient than a linear queue.

Data Structure Selection:-

A circular Queue is choosen because it efficiently utilizes available memory by wrapping around to the beginning of the queue when the rear reaches the last position.

Algorithm:

Enqueue:-

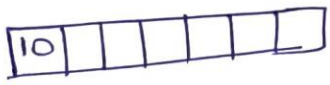
1) Check whether the queue is full.
2) If full, display Overflow.
3) Otherwise, move rear to the next circular position.
4) Insert the new element.

Dequeue:-

1) Check whether the queue is empty.
2) If empty, display underflow.
3) Remove the front elements.
4) Move front to the next circular position.

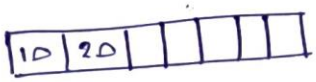
Step-By-Step Execution:-

Enqueue:- 10.



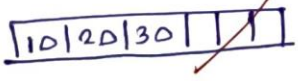
Front=0
Rare=0.

Enqueue:- 20.



Front=0
Rare=1.

Enqueue: 30.

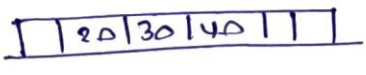


Front=0
Rare=2.

Dequeue: 10:-

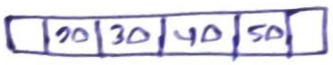
Front=1
Rear=2.

Enqueue: 40:



Front=1
rear=3.

Enqueue: 50

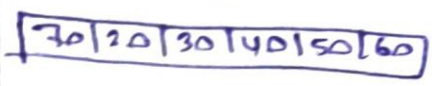


Front=1
Rear=4

Enqueue- 60:

Front=1
rear=5.

Enqueue-70:



Front=1
rear=0.

Advantages:-

- Efficient memory utilization.
- Reuses empty positions through wrap-around.
- No element shifting is required.
- Suitable for fixed-size buffers and scheduling applications.

Applications:-

- CPU scheduling.
- Printer queue management
- Network packet buffering
- keyboard input buffering.

Conclusion:-

A circular queue efficiently implements the FIFO principle while making optimal usse of memory. After executing the given operations, the queue contains 20, 30, 40, 50, 60 and 70, where front=1 and rear=0, demonstrating the wrap-around feature of the circular queue. Both enqueue and dequeue operations execute in O(1) time, making the circular queue suitable for applications in our daily life that has large data.